

Remote Patient Monitoring (RPM) System

Communication Architecture

1. Question

Design and explain the communication architecture of a Remote Patient Monitoring (RPM) system, covering its layers, protocols, and data flow from sensor to healthcare provider.

2. Definition

A Remote Patient Monitoring (RPM) Communication Architecture is the layered network and protocol framework that enables continuous or periodic acquisition of physiological data (e.g., ECG, SpO₂, blood pressure, glucose, temperature) from wearable or bedside sensors on a patient, and its reliable, secure, and timely transmission to a remote server or healthcare provider for storage, analysis, and clinical decision-making.

3. Description

The architecture is organized into four layers:

a) Sensing / Device Layer (Perception Layer)

- Wearable, implantable, or bedside biosensors (ECG, PPG, SpO₂, BP cuff, glucometer, temperature probe)
- Converts physiological signals into digital data
- Uses low-power microcontrollers (e.g., ARM Cortex-M, ESP32)

b) Gateway / Edge Layer

- Aggregates data from multiple sensors (via BLE, Zigbee, NFC)
- Performs edge processing: filtering, compression, anomaly pre-detection
- Devices: smartphone app, home hub, or dedicated IoT gateway

c) Network / Communication Layer

- Transmits data from gateway to cloud / hospital server
- Technologies: Wi-Fi, 4G/5G cellular, LoRaWAN (long-range, low-power), Ethernet (fixed hubs)
- Protocols: MQTT, CoAP, HTTP/HTTPS, WebSocket for real-time streaming

d) Application / Cloud Layer

- Cloud server or hospital EHR system stores and processes data
- Runs analytics and AI-based alerting, dashboards
- Sends alerts to clinicians via SMS, app notification, or EHR integration
- Ensures compliance with HIPAA / GDPR security standards

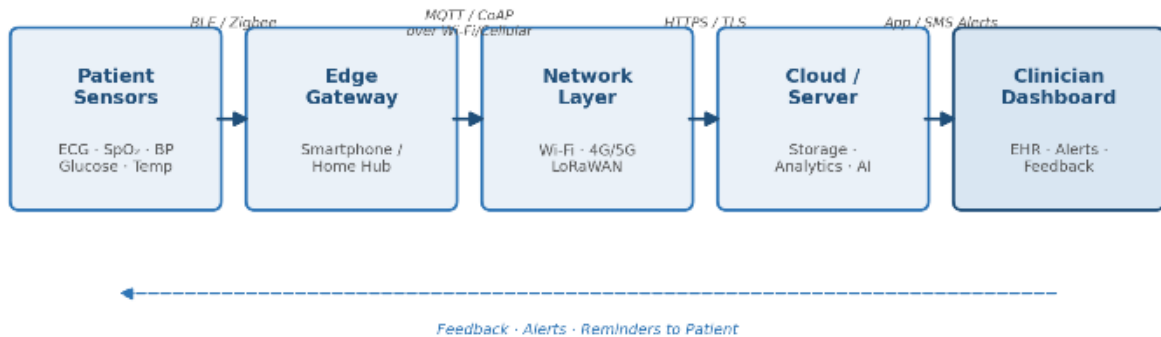
Data Flow

Stage	Description
-------	-------------

1. Patient Sensor	ECG, SpO2, BP, glucose, temperature sensors acquire raw physiological signals.
2. Short-Range Link	Bluetooth Low Energy (BLE) or Zigbee transmits data to a nearby gateway.
3. Edge Gateway	Smartphone or home hub aggregates, timestamps, compresses, and encrypts data.
4. Wide-Area Network	Wi-Fi / 4G-5G / LoRaWAN carries data from gateway to the internet.
5. Cloud Server / Database	Stores data, runs analytics and AI-based anomaly detection.
6. Clinician Dashboard / EHR	Visualizes trends, issues alerts, and integrates with hospital records.
7. Feedback to Patient	Alerts, medication reminders, or care instructions sent back to the patient.

4. Architecture Diagram

RPM System — Communication Architecture & Data Flow



5. Formula

a) Nyquist Sampling Rate

$$f_s \geq 2 f_{\max}$$

Minimum sampling rate to capture a signal without loss (e.g., ECG $\rightarrow f_s \geq 300$ Hz).

b) Channel Capacity (Shannon-Hartley Theorem)

$$C = B \log_2(1 + S/N)$$

C = channel capacity (bps), B = bandwidth (Hz), S/N = signal-to-noise ratio.

c) End-to-End Latency

$$T_{\text{total}} = T_{\text{sense}} + T_{\text{process}} + T_{\text{transmit}} + T_{\text{display}}$$

Sum of delays across the sensing-to-display pipeline.

d) Battery Life Estimation

$$L = C_{\text{battery}} / I_{\text{avg}}$$

C_{battery} = battery capacity (mAh), I_{avg} = average current drawn (mA).

6. Procedure

Step 1 – Requirement Analysis: Identify vital signs, sampling rate, and latency tolerance needed for monitoring.

Step 2 – Sensor & Edge Design: Select biosensors and low-power MCU; configure gateway to aggregate and encrypt data.

Step 3 – Network Selection: Choose BLE/Zigbee for short range and Wi-Fi/4G-5G/LoRaWAN for long range based on deployment.

Step 4 – Protocol & Cloud Setup: Implement MQTT/CoAP for transfer; deploy a HIPAA-compliant cloud backend with encryption (TLS/AES-256).

Step 5 – Analytics & Alerts: Apply threshold or ML-based anomaly detection to trigger clinician alerts via dashboard/EHR.

Step 6 – Testing & Validation: Validate latency, packet loss, battery life, and security under real-world conditions.

CODE:

```
% =====
% SIMULATION: Remote Patient Monitoring Communication Architecture
% Architecture: Sensor Nodes -> Wireless Channel -> IoT Gateway -> Cloud Server
% =====

clear; clc; close all;

%% 1. SYSTEM PARAMETERS & CONFIGURATION
fs = 100; % Sampling frequency (Hz)
duration = 10; % Simulation duration (seconds)
t = 0:1/fs:duration-1/fs; % Time vector
N = length(t);

num_patients = 3; % Number of patient sensor nodes
snr_dB = 15; % Channel Signal-to-Noise Ratio (dB)

%% 2. SENSOR NODE DATA GENERATION (Patient Terminals)
% Generate Synthetic Vital Signs for Patient Nodes
rng(42); % Set seed for reproducibility

% Node 1: Synthetic ECG Signal (~72 BPM with P-QRS-T components)
f_hr = 1.2; % Heart rate in Hz (~72 bpm)
ecg_clean = -0.15 * cos(2*pi*f_hr*t) ... % Baseline & P-wave
+ 1.2 * exp(-((mod(t, 1/f_hr) - 0.25)/0.02).^2) ... % QRS Complex
+ 0.25 * exp(-((mod(t, 1/f_hr) - 0.45)/0.08).^2); % T-wave
ecg_signal = ecg_clean + 0.05 * randn(1, N);

% Node 2: SpO2 Level (Pulse Oximetry around 98% with small fluctuations)
spo2_signal = 98 + 0.5 * sin(2*pi*0.1*t) + 0.2 * randn(1, N);

% Node 3: Body Temperature (in Celsius around 37.0 °C)
```

```
temp_signal = 37.0 + 0.1 * sin(2*pi*0.05*t) + 0.05 * randn(1, N);
```

```
%% 3. WIRELESS COMMUNICATION CHANNEL (Custom AWGN without Communications Toolbox)
add_awgn = @(sig, snr) sig + (sqrt((sum(abs(sig).^2)/length(sig)) / (10^(snr/10)))) * randn(size(sig));
```

```
ecg_tx = add_awgn(ecg_signal, snr_dB);
spo2_tx = add_awgn(spo2_signal, snr_dB);
temp_tx = add_awgn(temp_signal, snr_dB);
```

```
%% 4. IoT GATEWAY PROCESSING & ALERTS
% Gateway receives signals, filters noise, and checks for vital thresholds
```

```
% Digital Low-pass Filtering for ECG at Gateway
cutoff_freq = 35;
[b, a] = butter(4, cutoff_freq/(fs/2), 'low');
ecg_rx = filtfilt(b, a, ecg_tx);
```

```
% Moving Average Smoothing for SpO2 and Temperature
spo2_rx = movmean(spo2_tx, 10);
temp_rx = movmean(temp_tx, 10);
```

```
% Threshold Alert Logic
mean_spo2 = mean(spo2_rx);
mean_temp = mean(temp_rx);
```

```
fprintf('-- IoT GATEWAY HEALTH STATUS REPORT --\n');
fprintf('Patient 1 Mean SpO2: %.2f%%\n', mean_spo2);
fprintf('Patient 1 Mean Temp: %.2f °C\n', mean_temp);
```

```
if mean_spo2 < 95
    warning('ALERT: Low SpO2 level detected!');
else
    fprintf('SpO2 Status: NORMAL\n');
end
```

```
if mean_temp > 37.5
    warning('ALERT: High Temperature (Fever) detected!');
else
    fprintf('Temperature Status: NORMAL\n');
end
```

```
%% 5. DATA VISUALIZATION (Central Monitoring Dashboard)
figure('Name', 'Remote Patient Monitoring Dashboard', 'NumberTitle', 'off');
```

```
% Plot ECG Signal
subplot(3, 1, 1);
plot(t, ecg_signal, 'k-', 'LineWidth', 1); hold on;
plot(t, ecg_rx, 'r', 'LineWidth', 1.2);
title('Patient Node 1: ECG Signal Transmission');
xlabel('Time (s)'); ylabel('Voltage (mV)');
legend('Raw Sensor Data', 'Received & Filtered at Gateway');
grid on;
```

```
% Plot SpO2 Signal
subplot(3, 1, 2);
plot(t, spo2_tx, 'c', 'LineWidth', 0.8); hold on;
plot(t, spo2_rx, 'b', 'LineWidth', 1.5);
```

```

yline(95, 'r--', 'Critical SpO2 Limit (95%)');
title('Patient Node 2: SpO2 Level Monitoring');
xlabel('Time (s)'); ylabel('SpO2 (%)');
legend('Transmitted (Noisy)', 'Gateway Filtered', 'Location', 'best');
grid on;

```

```

% Plot Temperature Signal
subplot(3, 1, 3);
plot(t, temp_tx, 'm', 'LineWidth', 0.8); hold on;
plot(t, temp_rx, 'm', 'LineWidth', 1.5);
yline(37.5, 'r--', 'Fever Threshold (37.5°C)');
title('Patient Node 3: Body Temperature Monitoring');
xlabel('Time (s)'); ylabel('Temperature (°C)');
legend('Transmitted (Noisy)', 'Gateway Filtered', 'Location', 'best');
grid on;

```

OUTPUT:

